Difference Between Normal AI Agent & LangGraph

LangChain (Agent):

A reactive agent — gets a query, reasons, picks a tool, observes the result, and repeats until it has an answer. The agent loop (Reason → Act → Observe) is managed by LangChain internally. You define the tools and the prompt, LangChain handles the loop. Works great when the agent can solve the task in a single reasoning cycle without needing memory of past runs or complex branching.

LangGraph (Graph-based Agent):

A controlled, stateful agent — you explicitly define the nodes (steps) and edges (transitions) as a graph. The agent moves through the graph based on conditions, can loop back, branch, wait for human input, or persist state across runs.

Scenario:

An agent that handles customer refund requests. Simple cases ("what's my refund status?" → tool call → answer) work fine as a LangChain agent. But when the agent needs to verify identity → check policy → maybe escalate to human → wait for approval → loop back to ask the customer for more info → then process — that's a graph. The flow isn't a single ReAct cycle, it branches and loops based on what happens at each step. That's LangGraph.

When to Use Langchain vs LangGraph

When to go with LangChain Agent:

- Single agent, single task — "answer thisquestion using these tools"
- ReAct pattern is enough — reason, act,observe, done
- No need to pause, persist,

When to go with LangGraph Agent:

- You don't know the agent flow upfront — it needs to decidedynamically
- Agent needs to loop with self-correction ("that answerwasn't good enough, try again")
- Multi-agent handoff — one agent's output feeds intoanother based on conditions
- Long-running tasks — agent pauses, state persists, resumeshours later
- Error recovery — retry from the failed step, not from thebeginning

Core Capabilities of LangGraph

- **Durable execution** (survives failures)
- **Streaming** (real-time outputs and state updates)
- Human-in-the-loop workflows
- Stateful and long-running agents

Persistence in LangGraph

What is Persistence in LangGraph?

Persistence in LangGraph means saving the state of a graph while it is running, so that it can be:

- Recovered later
- Inspected by humans
- Replayed from any point
- Continued after failures

Without persistence:

- Every run is stateless
- Errors force a full restart
- No memory across interactions

With persistence:

- Graph state is saved step by step
- Long-running and interactive workflows become possible
- Time travel and durable execution come for free
- Persistence is the mechanism. Time Travel and Durable Execution are two powerful capabilities you get because of it.

Core Components of Persistence

LangGraph persistence is built using three key concepts:

- Threads
- Checkpoints
- Checkpointers

Together, they enable:

- Memory
- Human-in-the-loop workflows
- Time travel (replay)
- Fault tolerance (durable execution)

What is a Thread?

A **thread** is a **unique identifier** (thread_id) that represents:

- One continuous execution or conversation
- A sequence of graph runs sharing the same state

Key Points

- Every persisted graph run belongs to a thread
- All checkpoints are saved **inside a thread**
- Threads allow state access **after execution**

Example Use Case

- One chatbot conversation = one thread
- Multiple conversations = multiple threads

Why Thread ID is Mandatory

- When using persistence you **must** pass thread_id in config.

```
{"configurable": {"thread_id": "1"}}
```

WHY ?

- Checkpointers use thread_id as the **primary key.**
- Without it:
  - State cannot be saved
  - Execution cannot resume
  - Interrupts won’t work

What is a Checkpoint?

A **checkpoint** is a **snapshot of graph state** at a specific point in time.

When are Checkpoints Created?

- At **every super-step** of the graph
- Automatically when a checkpointer is enabled

What Does a Checkpoint Contain?

Each checkpoint stores a StateSnapshot object with:

- **config** – Configuration used for this step
- **metadata** – Execution details (step number, source node)
- **values** – Current state values
- **next** – A tuple of the node names to execute next in the graph.
- **tasks** – A tuple of Pregel Task objects that contain information about next tasks to be executed. If the step was previously attempted, it will include error information. If a graph was interrupted dynamically from within a node, tasks will contain additional data associated with interrupts.

Example for Checkpoints

Graph Flow :

```
START → node_a → node_b → END
```

Nodes:

- node_a updates state
- node_b updates state

With persistence enabled, 4 checkpoints are created:

- Initial empty state
- User input state
- After node_a
- After node_b (final state)

What is a Checkpointer?

A **checkpointer** is the component that:

- Saves checkpoints
- Loads checkpoints
- Lists checkpoint history

Role

- Enables persistence
- Enables recovery
- Enables replay and memory

Checkpointers - Examples in Action

1. Saves Checkpoints

- Stores state, metadata, and configuration
- Automatically saves after every super-step

```python
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
graph = workflow.compile(checkpointer=checkpointer)

# Checkpoint automatically saved after each node execution
result = graph.invoke(
    {"messages": "Hello"},
    config={"configurable": {"thread_id": "chat_1"}}
)
# ✓ State saved automatically at each step
```

Checkpointers - Examples in Action

2. **Loads Checkpoints**
   - Retrieves previous state using thread_id
   - Resumes execution from last checkpoint

```
config = {"configurable": {"thread_id": "chat_1"}}

result = graph.invoke(
    {"messages": "Continue conversation"},
    config=config
)
```

3. **Returns all checkpoints for a thread**
   - Stores state, metadata, and configuration
   - Automatically saves after every super-step

```
config = {"configurable": {"thread_id": "chat_1"}}

for state in graph.get_state_history(config):
    print(f"Step: {state.metadata['step']}")
    print(f"State: {state.values}")
```

State Management - Get State

What is get_state()?

A synchronous method that performs a read operation on the checkpoint store to retrieve the most recent StateSnapshot for a given thread identifier. It queries the checkpointer's storage backend and deserializes the persisted state into a runtime object.

```
config = {"configurable": {"thread_id": "conversation_1"}}
current_state = graph.get_state(config)
```

Core Features

- Returns immutable StateSnapshot object from checkpoint storage
- Automatically saves after every super-step
- Debug graph execution by viewing intermediate states
- Get state for specific conversation threads

State Management - Update State

Overview

A method that manually modifies the graph state at any checkpoint without re-executing nodes. It applies the given values to the state as if they were generated by a specified node

```
new_config = graph.update_state(
    config,
    values={"messages": updated_messages},
    as_node="chatbot"
)
```

Method Parameters:

- **Config -** Configuration with `thread_id` and optional `checkpoint_id`
- **values** - State dictionary processed through channel reducers
- **as_node** - Node attribution for update and execution control

Core Features

- Returns immutable StateSnapshot object from checkpoint storage
- Supports `as_node` parameter to control next node execution
- Returns new configuration with updated checkpoint_id

State Management - Replay

Overview

A mechanism to replay prior graph execution by invoking the graph with both `thread_id` and `checkpoint_id`. LangGraph re-plays previously executed steps before the checkpoint and executes steps after the checkpoint as a new fork.

```
config = {
    "configurable": {
        "thread_id": "1",
        "checkpoint_id": "0c62ca34-ac19-445d-bbb0-5b4984975b2a"
    }
}
await graph.invoke(None, config)
```

Core Features

- Re-plays steps before specified checkpoint without re-execution
- **Step Tracking** - LangGraph knows which steps were previously executed
- All steps after checkpoint_id create new execution branch

Persistence - Overview

